

プログラミングI 演習

Computer Programming I: Exercises

第10回 文字列と文字列操作

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

本日の講義

本日の目標

- コンピュータ内部での文字の表現方法を知ろう
- 文字の入力や出力ができるようになるろう
- 文字や文字列を用いたプログラムを書けるようになるろう

本日の資料

1. 前回のおさらい
2. 文字列
3. 文字列操作
4. 演習課題

1. 前回のおさらい

C言語の大事な概念で、身に着けると出来ることが増えます

- ポインタ/ポインタ変数: 「アドレス」を保存するための変数
- 関数に配列を引数として渡すとき, 「値渡し」「ポインタ渡し」の違いを知っておく必要がある

ポインタを使うことの代表的な利点

- 関数から戻値を(実質)複数受け取れる
- 関数に複数のデータを(=配列やファイルを)渡せる
- 動的メモリ割当てが可能になる

本演習で学ぶこと

- ポインタ変数の宣言・読み書きのための文法
- ポインタ渡しの文法
- 「値渡し」と「ポインタ渡し」の違い

1. 前回のおさらい

配列を引数とする関数(演習8-4[1]解答例)

```
#include <stdio.h>
#define N 10
```

1次配列なら要素数は省略可

```
void Scalar(int a[], int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
```

配列がポインタ渡しになっているのなら、
「&」を使っても表現可能？

```
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;
```

```
    Scalar(a, k);
```

配列を渡すときは配列名(=同名のポインタ)
だけで良かった

```
    for(i=0; i<N; i++) printf("a[%d]は%d¥n", i, a[i]);
    return 0;
```

```
}
```

1. 前回のおさらい

正しい！ → &を使って書き換えてみる

```
#include <stdio.h>
#define N 10
```

```
void Scalar(int a[], int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
```

```
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;
```

```
    Scalar(&a[0], k);
```

```
    for(i=0; i<N; i++) printf("a[%d]は%d¥n", i, a[i]);
    return 0;
```

```
}
```

では、こちらもポインタ変数として表現することも可能？

このように変えても同じ結果が得られる

1. 前回のおさらい

それも正しい！ → *を使って書き換えてみる

```
#include <stdio.h>
#define N 10
```

このように変えても同じ結果が得られる

```
void Scalar(int *a, int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
```

書き方が複数あるため、混乱しやすい
→ 今後も出てくるので覚えておきましょう

```
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;

    Scalar(&a[0], k);

    for(i=0; i<N; i++) printf("a[%d]は%d¥n", i, a[i]);
    return 0;
}
```

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d, z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

最終的に, x, y, z[0], z[1], z[2]には何が入るでしょうか.
問題を解いた後に実行してみよう

1. 前回のおさらい

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	
107	*q	

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```


1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

c[]は*cに変換可能

zは「&z[0]」に変換可能

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	1
202	*b	102
203	*c(c[0])	103
204	*d	104
205	*e	105

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	1
202	*b	102
203	*c(c[0])	103
204	*d	104
205	*e	105
206	tmp	

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2->1
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	1->2
202	*b	102
203	*c(c[0])	103
204	*d	104
205	*e	105
206	tmp	1

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	1
103	z[0]	3->4
104	z[1]	4->3
105	z[2]	5
106	*p	104
107	*q	105
201	a	2
202	*b	102
203	*c(c[0])	103
204	*d	104
205	*e	105
206	tmp	1->3

c[1]は *(&c[0] + 1)と書き換えられます

c[0]のアドレスは103なので、*(&c[0] + 1)はアドレスが104のところの値

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	1
103	z[0]	4
104	z[1]	3->5
105	z[2]	5->3
106	*p	104
107	*q	105
201	a	2
202	*b	102
203	*c(c[0])	103
204	*d	104
205	*e	105
206	tmp	3->3

1. 前回のおさらい

演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

x= 1, y= 1, z[0]= 4, z[1]= 5, z[2]= 3 となる

アドレス	変数名	値
101	x	1
102	y	1
103	z[0]	4
104	z[1]	5
105	z[2]	3
106	*p	104
107	*q	105
201	a	2
202	*b	102
203	*c(c[0])	103
205	*d	104
206	*e	105
207	tmp	3

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[]){
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d, z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

最終的に, x, y, z[0], z[1], z[2]には何が入るでしょうか.
問題を解いた後に実行してみよう(test関数が少し変化)

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アド レス	変数 名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	
107	*q	

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	
202	*b	
203	*c	
204	*d	
205	*e	

b[]は*bに変換可能だった
c[], d[], e[]も同様

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	1
202	*b	102
203	*c	103
204	*d	104
205	*e	105

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	1
202	*b	102
203	*c	103
204	*d	104
205	*e	105
206	tmp	

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	2->1
103	z[0]	3
104	z[1]	4
105	z[2]	5
106	*p	104
107	*q	105
201	a	1->2
202	*b	102
203	*c	103
204	*d	104
205	*e	105
206	tmp	1

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	1
103	z[0]	3->5
104	z[1]	4
105	z[2]	5->3
106	*p	104
107	*q	105
201	a	2
202	*b	102
203	*c	103
204	*d	104
205	*e	105
206	tmp	1->5

c[2]は *(&c[0] + 2)と書き換えられます

c[0]のアドレスは103なので、*(&c[0] + 2)はアドレスが105のところの値

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

アドレス	変数名	値
101	x	1
102	y	1
103	z[0]	5
104	z[1]	4->3
105	z[2]	3->3
106	*p	104
107	*q	105
201	a	2
202	*b	102
203	*c	103
204	*d	104
205	*e	105
206	tmp	5->3

d[1]は *(&d[0] + 1)と書き換えられます

d[0]のアドレスは104なので, *(&d[0] + 1)はアドレスが105のところの値

1. 前回のおさらい

演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[])
{
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void) {
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d,
           z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

x= 1, y= 1, z[0]= 5, z[1]= 3, z[2]= 3 となる

アドレス	変数名	値
101	x	1
102	y	1
103	z[0]	5
104	z[1]	3
105	z[2]	3
106	*p	104
107	*q	105
201	a	2
202	*b	102
203	*c	103
204	*d	104
205	*e	105
206	tmp	3

1. 前回のおさらい

演習9-4 アドレスの足し算①(空白数カウンタ)

文字列中の空白文字' 'の数を数える関数whcountを完成させよ。
例えば, "I am a student of TUAT."という文字列に対して, 5を返すものとする。
また, 空白文字が2文字以上連続している場合, それぞれ1文字として数える。

ヒント

```
#include <stdio.h>

_____ whcount(char *s){
    int c = _____;
    while(*s != _____){
        if(*s == _____) c++;
        s++;
    }
    return _____;
}

int main(void){
    int count;
    char String[40]="I am a student of TUAT";
    count = whcount(String);
    printf("空白の数は%dです\n", count);
    return 0;
}
```

1. 前回のおさらい

演習9-4 アドレスの足し算①(空白数カウンタ)

文字列中の空白文字' 'の数を数える関数whcountを完成させよ。
例えば, "I am a student of TUAT."という文字列に対して, 5を返すものとする。
また, 空白文字が2文字以上連続している場合, それぞれ1文字として数える。

ヒント

```
#include <stdio.h>
int whcount(char *s){
    int c = 0;
    while(*s != '\0'){
        if(*s == ' '){ c++; }
        s++;
    }
    return c;
}
int main(void){
    int count;
    char String[40]="I am a student of TUAT";
    count = whcount(String);
    printf("空白の数は%dです\n", count);
    return 0;
}
```

文字列は
終端文字で終わる

空白ならカウントcを1つ増加

文字のアドレスを1つ進める

1. 前回のおさらい

発展9-1 アドレスの足し算②(英単語数カウンタ)

文字列中の英単語の数を数える関数wdcountを完成させよ。
例えば, "I am a student of TUAT."という文字列に対して, 6を返すものとする。
ただし, 空白は半角スペース2つ以上でも良い(全角スペースは入らない)

ヒント

```
#include <stdio.h>

_____ wdcount(char *s){
    int c = _____;
    while(*s != _____){
        while(*s == _____) s++;
        if(*s != _____) c++;
        while((*s != ' ') _____ (*s != '\0')) s++;
    }
    return _____;
}

int main(void){
    int count;
    char String[40]="I am a student of TUAT";
    count = wdcount(String);
    printf("英単語の数は%dです\n", count);
    return 0;
}
```

赤下線部
は演習9-4
と同じもの
が入る

1. 前回のおさらい

発展9-1 アドレスの足し算②(英単語数カウンタ)

文字列中の英単語の数を数える関数wdcountを完成させよ。
例えば, "I am a student of TUAT."という文字列に対して, 6を返すものとする。
ただし, 空白は半角スペース2つ以上でも良い(全角スペースは入らない)

ヒント

```
#include <stdio.h>
int wdcount(char *s){
    int c = 0;
    while(*s != '\0'){
        while(*s == ' ') { s++; }
        if(*s != '\0') { c++; }
        while((*s != ' ') && (*s != '\0')) { s++; }
    }
    return c;
}

int main(void){
    int count;
    char String[40]="I am a student of TUAT";
    count = wdcount(String);
    printf("英単語の数は%dです\n", count);
    return 0;
}
```

赤下線部
は演習9-4
と同じもの
が入る

空白の間は文字のアドレス
を1つ進める

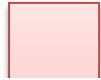
単語の先頭でカウンタcを
1つ増加

空白以外, かつ, 終端以外の場合,
すなわち単語の場合には文字のアド
レスを1つ進める

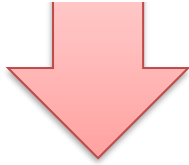
2. 文字列

計算機の中の文字

計算機の中には数値しか存在しない



人間にとっては、文字も扱いたい



文字(画像データ)に番号(数値)を割り振って管理する仕組みを作ればいい

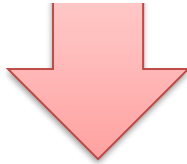
2. 文字列

計算機の中の文字

計算機の中には数値しか存在しない



人間にとっては、文字も扱いたい



文字(画像データ)に番号(数値)を割り振って管理する仕組みを作ればいい

文字に割り振られた番号を「文字コード」と呼ぶ

ここでは、基本的な文字コードの集合であるJIS X 0201コードを扱う

JIS X 0201 文字コード表

文字	10進	文字	10進
+	43	W	87
,	44	X	88
-	45	Y	89
.	46	Z	90
/	47	[	91
0	48	¥	92
1	49	]	93

キーボードに書かれているような、
いわゆる半角文字

JIS X 0201

上位4bit

下位4bit



例 'A'=0x41 '1'=0x31 '?'=0x3F

※0xは右に続く数字が16進数表示であることを表わす.

※文字の'1'と数字の1は異なる!

上位4bit

下位4bit

	00	10	20	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0
00		DE		0	@	P		p				一	タ	ミ		
01	SH	D1	!	1	A	Q	a	q			。	ア	チ	ム		
02	SX	D2	"	2	B	R	b	r			「	イ	ツ	メ		
03	EX	D3	#	3	C	S	c	s			」	ウ	テ	モ		
04	ET	D4	\$	4	D	T	d	t			、	エ	ト	ヤ		
05	EQ	NK	%	5	E	U	e	u			・	オ	ナ	ユ		
06	AK	SN	&	6	F	V	f	v			ヲ	カ	ニ	ヨ		
07	BL	EB	'	7	G	W	g	w			ァ	キ	ヌ	ラ		
08	BS	CN	(	8	H	X	h	x			ィ	ク	ネ	リ		
09	HT	EM	)	9	I	Y	i	y			ゥ	ケ	ノ	ル		
0A	LF	SB	*	:	J	Z	j	z			ェ	コ	ハ	レ		
0B	HM	EC	+	;	K	[	k	{			ォ	サ	ヒ	ロ		
0C	CL	→	,	<	L	¥	l				ャ	シ	フ	ワ		
0D	CR	←	—	=	M	]	m	}			ュ	ス	ヘ	ン		
0E	SO	↑	.	>	N	^	n	—			ョ	セ	ホ	ゝ		
0F	SI	↓	/	?	O	_	o				ッ	ソ	マ	。		

2. 文字列

文字を表示するプログラム

```
#include <stdio.h>
int main(void) {
    int n=65;
    printf("数値では%d¥n",n);
    printf("文字では%c¥n",n);
    return 0;
}
```

関数「printf」では、「%c」(**C**haracter)と書くことで、引数の値を文字として扱う

ここでは、65番の文字である「A」が表示された

実行結果

```
数値では65
文字ではA
```

2. 文字列

char型

- JIS X 0201の前半の文字は1バイトのサイズの文字用の型で表現可能
→char型
- -128~127の数値を扱える

char型を使ったプログラム

```
#include <stdio.h>
int main(void) {
    char n=65;
    printf("数値では%d¥n",n);
    printf("文字では%c¥n",n);
    return 0;
}
```

実行結果

数値では65
文字ではA

2. 文字列

プログラム中の文字データ

char型を使ったプログラム

```
#include <stdio.h>
int main(void) {
    char n=65;
    printf("数値では%d¥n",n);
    printf("文字では%c¥n",n);
    return 0;
}
```

文字を表示させるために数値を代入しているが、これではプログラムが何をしているのか分かりにくい

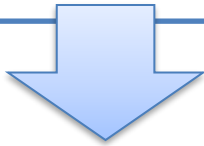
2. 文字列

プログラム中の文字データ

char型を使ったプログラム

```
#include <stdio.h>
int main(void) {
    char n=65;
    printf("数値では%d¥n",n);
    printf("文字では%c¥n",n);
    return 0;
}
```

文字を表示させるために数値を代入しているが、これではプログラムが何をしているのか分かりにくい



```
#include <stdio.h>
int main(void) {
    char n='A';
    printf("数値では%d¥n",n);
    printf("文字では%c¥n",n);
    return 0;
}
```

「」（シングルクォート）で1文字を囲むことで、「これは文字データである」ということを表せる

実質的には、プログラムが「65という数値」を変数「n」に代入することは変わらない

2. 文字列

文字列

数列が「int型の配列」で表せるのだから……
文字列は「char型の配列」として実現できる

char型の配列を使ったプログラム

```
#include <stdio.h>
int main(void) {
    char String[20]={'H','e','l','l','o','W','o','r','l','d','\0'};
    int i;
    for(i=0;i<20;i++)
        printf("%c",String[i]);
    printf("\n");
    return 0;
}
```

実行結果

HelloWorld

2. 文字列

文字列

数列が「int型の配列」で表せるのだから……
文字列は「char型の配列」として実現できる

char型の配列を使ったプログラム

```
#include <stdio.h>
int main(void) {
    char String[20]={'H','e','l','l','o','W','o','r','l','d','\0'};
    int i;
    for(i=0;i<20;i++)
        printf("%c",String[i]);
    printf("\n");
    return 0;
}
```

実行結果

HelloWorld



毎回文字の長さに応じた処理を作っていない上、このコードでは文字列の終端のヌル文字（'\0'）以降も強制的に出力してしまう。初期化されていない配列の場合には入っている値が不明で、それまで表示される点で良くない。

2. 文字列

終端文字「¥0」(数字のゼロ)

文字列の最後であることを表す
「特殊文字」(他にも「¥n:改行」、「¥t:タブ」等がある)



文字列の中に「¥0」が現れるまで表示(何らかの処理)を行うプログラムを書けば、文字列の扱いが楽になる

```
#include <stdio.h>
int main(void) {
    char String[20]={'H','e','l','l','o','¥0'};
    int i = 0;
    while(String[i] != '¥0')
        printf("%c",String[i++]);
    printf("¥n");
    return 0;
}
```

これで、どんな長さの文字列
が来ても正しい表示が可能
になる

String[i]を表示してから、i++を実行

2. 文字列

関数「printf」での文字列表示

```
#include <stdio.h>
int main(void) {
    char String[20]={'H','e','l','l','o','¥0'};

    printf("%s¥n",String);
    return 0;
}
```

「%s」の位置に文字列が表示される
引数には配列名を渡す

関数「printf」の中では、先程のプログラムに近い処理が行われる

2. 文字列

関数「printf」での文字列表示

```
#include <stdio.h>
int main(void) {
    char String[20]={'H','e','l','l','o','¥0'};

    printf("%s¥n",String);
    return 0;
}
```

「%s」の位置に文字列が表示される
引数には配列を渡す

関数「printf」の中では、先程のプログラムに近い処理が行われる

文字列の初期化

```
#include <stdio.h>
int main(void) {
    char String[20] = "Hello";

    printf("%s¥n",String);
    return 0;
}
```

ダブルクォートで囲まれた複数の文字
は、文字列として配列に格納される

終端記号「¥0」も自動的に付加される

終端記号より後の値も「¥0」が入る

2. 文字列

文字列の入力 (第9回のスライドより)

```
#include <stdio.h>
int main(void) {
    char String[20];
    scanf("%19s",String);
    printf("%s\n",String);
    return 0;
}
```

配列名が先頭のアドレスを記録したポインタを表すので'&'をつけなくて良い

printfも%sなら配列名を引数に書く
(第10回で説明)
→処理が分かりましたか？

関数「scanf」も「%s」を用いて文字列を入力することができる。

(注: 配列の要素数より一つ少ない数を用いて%19sとすることでユーザの入力文字数が宣言した配列の要素数以上になった場合のバッファオーバーランを防ぐことができる)

2. 文字列

文字列の入力

```
#include <stdio.h>
int main(void) {
    char String[20];
    scanf("%19s",String);
    printf("%s¥n",String);
    Return 0;
}
```

関数「scanf」も「%s」を用いて文字列を入力することができる。

(注: 配列の要素数より一つ少ない数を用いて%19sとすることでユーザの入力文字数が宣言した配列の要素数以上になった場合のバッファオーバーランを防ぐことができる)

scanf("%19s",String); の中で行われていること

キーボードからの値「char Keyboard[128]」

l	n	p	u	t	¥0	??
---	---	---	---	---	----	----

Stringの状態

??	??	??	??	??	??	??
----	----	----	----	----	----	----

scanf関数

```
{
    int i = 0;
    while(String[i] != '¥0'){
        String[i] =Keyboard[i];
        i++;
    }
}
```

2. 文字列

文字列の入力

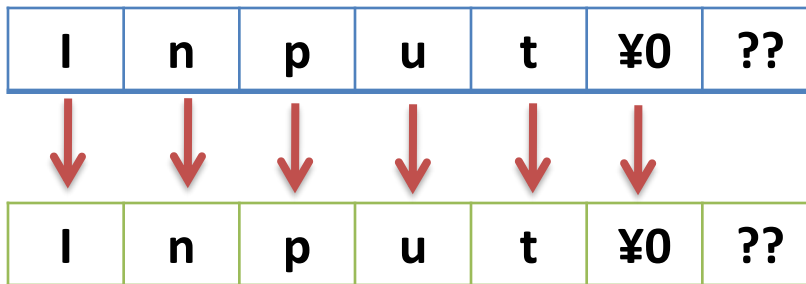
```
#include <stdio.h>
int main(void) {
    char String[20];
    scanf("%19s", String);
    printf("%s¥n", String);
    Return 0;
}
```

関数「scanf」も「%s」を用いて文字列を入力することができる。

(注: 配列の要素数より一つ少ない数を用いて%19sとすることでユーザの入力文字数が宣言した配列の要素数以上になった場合のバッファオーバーランを防ぐことができる)

scanf("%19s", String); の中で行われていること

キーボードからの値「char Keyboard[128]」



scanf関数

```
{
    int i = 0;
    while(String[i] != '¥0'){
        String[i] = Keyboard[i];
        i++;
    }
}
```

2. 文字列

例： 小文字を大文字に変換する

```
#include <stdio.h>
int main(void) {
    char String[20];
    int i=0;

    scanf("%19s", String);

    while(String[i] != '\0')
    {
        if(String[i] >= 'a' && String[i] <= 'z')
            String[i] = String[i] + ('A' - 'a');
        i++;
    }

    printf("%s\n", String);
    return 0;
}
```

• 文字列の場合は終端記号「¥0」があるので、終わりも判別できる

- JIS X 0201コードでは、アルファベットに割り当てられた数値は連続している、ということを利用
- 文字列「string」の「i」番目の文字が、「a~z」の範囲にあった場合大文字に変換する
(「A」と「a」の位置の差を足す)

2. 文字列

例： 多次元配列の利用

```
#include <stdio.h>
int main(void) {
    char NameList[5][2][20];
    int i;
    for(i=0;i<5;i++)
    {
        printf("名前の入力:");
        scanf("%19s %19s", NameList[i][0],
              NameList[i][1]);
    }

    for(i=0;i<5;i++)
    {
        printf("%s %s¥n", NameList[i][1],
              NameList[i][0]);
    }
    return 0;
}
```

例：5人分の名前（姓・名）を扱うプログラム

3次元配列を利用して
[番号][姓・名][名前文字列]
と表現した

前スライドの1次元配列の場合、添え字を削除して入力。
多次元配列の場合、**最後の添え字のみを削除**して入力
※理由は次のスライド参照

2. 文字列

例： 2次元配列の例 (教科書5.4.2, 5.4.3参照)

- char bloodtype[4][3] = {"A", "B", "AB", "O"};

例えば bloodtype[2]は{'A', 'B', '¥0'}の先頭のアドレスを表す

[0][0]	[0][1]	[0][2]	bloodtype[0]	A	¥0	
[1][0]	[1][1]	[1][2]	bloodtype[1]	B	¥0	
[2][0]	[2][1]	[2][2]	bloodtype[2]	A	B	¥0
[3][0]	[3][1]	[3][2]	bloodtype[3]	O	¥0	

3. 文字列操作

C言語のライブラリに用意されている関数

※「#include <string.h>」を先頭に追加すること

文字列の長さを返す

```
int strlen(char *str)
```

プログラム例:

```
char str[64] = "Test_String";  
int Length = strlen(str);  
  
printf("文字列「%s」の長さは%d",str, Length);
```

実行結果:

文字列「 Test_String 」の長さは11

文字列をコピーする

```
Char *strcpy(char *str1,char *str2)
```

プログラム例:

```
char str1[64] = "Test";  
char str2[64] = "String";  
char *str3 = strcpy(str1,str2);  
printf("%s:%s",str1,str3);
```

実行結果:

String:String

*str2を*str1にコピー
→*str1に「String」が入る
その*str1を*str3に代入

3. 文字列操作

C言語のライブラリに用意されている関数

※「#include <string.h>」を先頭に追加すること

文字列同士を比較する

```
int strcmp(char *str1, char *str2)
```

プログラム例:

```
char str1[64] = "abcdA";  
char str2[64] = "abcdZ";  
int comp = strcmp(str1, str2);  
printf("%d", comp);
```

実行結果:

-25

strcmp関数は文字列を比較し,
(第1引数) > (第2引数)なら **正数**
(第1引数) = (第2引数)なら **0**
(第1引数) < (第2引数)なら **負数**
を返す. 大小は内部値で決まる

```
char str1[64] = "abcdA";  
char str2[64] = "abcdA";  
int comp = strcmp(str1, str2);  
printf("%d", comp);
```

実行結果:

0

3. 文字列操作

C言語のライブラリに用意されている関数

※「#include <string.h>」を先頭に追加すること

文字列を追加する

Char ***strcat**(char *str1, char *str2)

プログラム例:

```
char str1[64] = "Test";  
char str2[64] = "String";  
char *str3 = strcat(str1, str2);  
printf("%s:%s", str1, str3);
```

str2に格納されている文字列をstr1の後に追加する
→ これにより str1には「TestString」が格納される

そのstr1をstr3に代入している

実行結果:

TestString:TestString

ここから演習課題です

4. 演習課題

演習10-1 文字表現の確認

```
#include<stdio.h>

int main(void)
{
    int n=65;
    printf("数値では%d¥n", n) ;
    printf("文字では%c¥n", n) ;
    return 0;
}
```

このとおりプログラムし実行してみよう。
また、(1) nの値を変更してみよう。
(2) 「int n=65」を「char n=65」や
「char n='A'」としてみよう

4. 演習課題

演習10-2 文字列の出力

```
#include<stdio.h>

int main(void)
{
    char String[20]={'H','e','l','l','o','¥0'};
    int i = 0;
    while(String[i] != '¥0')
        printf("%c",String[i++]);
    printf("¥n");
    return 0;
}
```

このとおりプログラムし実行してみよう。
また、

(1) 初期値の文字数を変えてみよう(上記は5文字)

(2) 初期設定を「char String[20] = "Hello"」として実行してみよう

4. 演習課題

演習10-3 文字列の入力

```
#include<stdio.h>
int main(void)
{
    char String[20];
    int i=0;

    scanf("%19s", String);

    while(String[i] != '\0'){
        if(String[i] >= 'a' && String[i] <= 'z')
            String[i] = String[i] + ('A' - 'a');
        i++;
    }

    printf("%s\n", String);
    return 0;
}
```

このとおりプログラムし実行し、小文字が大文字に変わることを確認しよう。

また、**大文字をすべて小文字にする**プログラムに書き換えてみよう

JIS X 0201コードを確認して、この操作の意味を確認してみてください。文字を数値に変換して考えてみよう

4. 演習課題

演習10-4 第2回レポート課題(次回説明)に使えるプログラム

```
#include<stdio.h>
int main(void)
{
    int i,j;
    char NameList[2][2][51];      /* 1つめは名前ID, 2つめは情報ID, 3つめは文字列入力用 */
    for(i=0; i<=1; i++){
        for(j=0; j<=1; j++){
            if(j==0){
                printf("ID: %dの名前を入力(全角25文字まで)", i);
                scanf("%50s", NameList[i][j]);
            }else{
                printf("ID: %dの住所を入力(全角25文字まで)", i);
                scanf("%50s", NameList[i][j]);
            }
        }
    }
    for(i=0; i<=1; i++){
        for(j=0; j<=1; j++){
            if(j==0){
                printf("ID: %dの名前は%s¥n", i, NameList[i][j]);
            }else{
                printf("ID: %dの住所は%s¥n", i, NameList[i][j]);
            }
        }
    }
    return 0;
}
```

このとおりプログラムし実行してみよう.

日本語(2バイト文字)は(Altキー+)全角半角キー でできるはず…

4. 演習課題

演習10-5 パスワードチェック機能の作成

1. 入力されたパスワードが「良いパスワード」か否かを判定する
2. 「良いパスワード」と判定された場合、(確認のため)再入力を求める

良いパスワードの条件(調べればもっと良い条件がありますが、この演習では下記とします)

1. 半角で8文字以上
2. 数字(0~9), 小文字(a~z), 大文字(A~Z)のすべての種類が使われている. (例: abcdef → ×, abcdefGH → ×, abCDef43 → ○)

ヒント

- 入力文字数チェックは, strlen関数を使うと良い.
- 数字, 小文字, 大文字のチェックは演習10-3等を参考にすると良い.
- 再入力したものが最初と同じかどうかは, strcmp関数を使ってチェックすると良い.

4. 演習課題

発展10-1 名簿の整理プログラム

要求される機能

1. 5人分の姓・名を入力する（姓→名の順で入力するようにし、姓名間に1つ半角スペースをいれる。5人の姓名はプログラム中で設定しても良い）
2. 出力用に書式を揃えて表示する
→ 書式: 名（先頭は大文字, 他は小文字）（半角スペース）姓（全て大文字）
例: 入力=kOnDo saToshi → 出力= Satoshi KONDO

必要になりそうな機能

1. 「姓」文字列をすべて大文字にする
2. 「名」文字列をすべて小文字にする。その後、先頭だけ大文字にする
3. 「名」から出力する

本日の説明内容はおしまいです。
課題に取り組んでください。